

Univerzitet u Beogradu
Fakultet organizacionih nauka

Master akademske studije
Studijski program: Elektronsko poslovanje
Modul: Tehnologije elektronskog poslovanja

Projektna dokumentacija

**AR pametne učionice — prikaz kontekstualnih
informacija iz pametnog okruženja u proširenoj
stvarnosti**

Studenti:

Ivan Jelisavčić

Broj indeksa: 2025/3263

Mateja Đurašić

Broj indeksa: 2025/3271

Jun 2026.

Sadržaj

1	Uvod u proširenu stvarnost	2
2	ARCore platforma	3
2.1	Praćenje pokreta	4
2.2	Razumevanje okruženja	4
2.3	Procena osvetljenja	5
2.4	Dubina, sidra i objekti praćenja	6
2.5	Praćenje slika (Augmented Images)	6
3	Arhitektura sistema	7
3.1	Serverska aplikacija	8
3.2	Kontrolna tabla	9
3.3	AR aplikacija	10
4	Razvoj ARCore aplikacije	10
4.1	Potrebna oprema i softver na računaru	11
4.2	Potrebna oprema na telefonu	11
4.3	Kreiranje projekta i zavisnosti	12
4.4	Podešavanje manifesta	14
4.5	Koraci do prve AR scene	15
5	Implementacija AR pregleda učionica	18
5.1	Konfiguracija sesije i baze slika	18
5.2	Prepoznavanje učionice	19
5.3	Sidrenje panela	20
5.4	Iscrtavanje panela	21
5.5	Preuzimanje podataka u realnom vremenu	21
5.6	Životni ciklus i oslobađanje resursa	23
6	Prikaz aplikacije	23
6.1	Početni ekran	23
6.2	Ekran sa uputstvom	24
6.3	AR ekran — prepoznavanje markera	25
6.4	AR ekran — usidreni panel	25
6.5	Demonstracija sa kontrolnom tablom	26

1 Uvod u proširenu stvarnost

Proširena stvarnost (engl. *Augmented Reality*, skraćeno AR) predstavlja tehnologiju koja digitalni sadržaj — trodimenzionalne modele, tekst, slike ili animacije — projektuje preko slike stvarnog sveta u realnom vremenu. Za razliku od klasičnih računarskih prikaza, koji postoje isključivo na ekranu i odvojeno od fizičkog okruženja, proširena stvarnost digitalne elemente uklapa u prostor oko korisnika tako da oni deluju kao njegov sastavni deo: virtuelna fotelja stoji na stvarnom podu, digitalna strelica pokazuje duž stvarne ulice, a informaciona tabla lebdi iznad stvarnih vrata učionice. Ključna odlika ove tehnologije jeste interaktivnost u realnom vremenu — kako se korisnik kreće kroz prostor, digitalni sadržaj se kontinuirano prilagođava novoj perspektivi, čime se stvara utisak da virtuelni objekti zaista postoje u fizičkom svetu.

Proširenu stvarnost treba jasno razlikovati od virtuelne stvarnosti (engl. *Virtual Reality*). Virtuelna stvarnost korisnika u potpunosti izmešta u sintetičko okruženje: stavljanjem namen-skih naočara stvarni svet nestaje, a zamenjuje ga računarski generisana scena. Proširena stvarnost, nasuprot tome, stvarni svet zadržava kao osnovu i samo ga dopunjuje. Ovaj odnos slikovito opisuje kontinuum stvarnosti i virtuelnosti — zamišljena osa na čijem jednom kraju stoji potpuno stvarno okruženje, a na drugom potpuno virtuelno. Proširena stvarnost nalazi se bliže stvarnom kraju kontinuumu, jer virtuelni elementi čine manji deo doživljaja, dok virtuelna stvarnost zauzima suprotni pol. Između njih se prostiru različiti oblici mešane stvarnosti, u kojima se udeo digitalnog i fizičkog sadržaja postepeno menja.

Iako se proširena stvarnost doživljava kao savremena tehnologija, njeni koreni sežu decenijama unazad. Još krajem šezdesetih godina dvadesetog veka konstruisani su prvi uređaji koji su se nosili na glavi i pred oči korisnika postavljali jednostavnu računarsku grafiku uklopljenu u pogled na prostoriju. Ti rani sistemi bili su glomazni, vezani kablom za računar i dostupni isključivo u laboratorijama. Tokom devedesetih godina tehnologija je našla primenu u vojnoj avijaciji i industrijskim istraživanjima, a sam termin „proširena stvarnost” ušao je u širu upotrebu. Prava prekretnica, međutim, nastupila je pojavom pametnih telefona: uređaj koji milijarde ljudi svakodnevno nose u džepu poseduje upravo ono što je proširenoj stvarnosti potrebno — kameru, ekran, senzore pokreta i dovoljno snažan procesor. Tehnologija koja je nekada zahtevala specijalizovanu i skupu opremu postala je dostupna svakom vlasniku prosečnog mobilnog telefona.

Široka dostupnost otvorila je vrata izuzetno raznovrsnim primenama. U oblasti igara i zabave globalni fenomen postala je igra *Pokémon GO*, koja je virtuelna bića smestila u parkove, ulice i trgove stvarnih gradova i time milionima korisnika po prvi put približila ideju proširene stvarnosti. U trgovini i opremanju doma aplikacije poput *IKEA Place* omogućavaju da se komad nameštaja u prirodnoj veličini „postavi” u dnevnu sobu pre kupovine, pa kupac odmah vidi da li se nova sofa uklapa u prostor i bojom i dimenzijama. U navigaciji se uputstva za kretanje iscertavaju direktno preko slike ulice koju kamera snima, tako da pešak umesto apstraktne mape vidi strelicu koja lebdi tačno iznad raskrsnice na kojoj treba da skrene.

Obrazovanje je još jedno područje u kojem proširena stvarnost pokazuje punu vrednost: umesto statičnih ilustracija u udžbeniku, učenici mogu da obišu trodimenzionalni model ljudskog srca koji kuca na školskoj klupi ili da posmatraju Sunčev sistem kako kruži iznad katedre. U medicini se tehnologija koristi za vizuelizaciju vena prilikom davanja injekcija, kao i za navođenje hirurga tokom zahvata, pri čemu se snimci unutrašnjih organa projektuju direktno preko tela pacijenta. U industriji i održavanju, tehničar koji popravlja složenu mašinu može kroz ekran ili pametne naočare da vidi uputstva za sklapanje iscertana tačno preko delova na koje se odnose, čime se smanjuje broj grešaka i skraćuje obuka.

Pored ovih specijalizovanih domena, proširena stvarnost nezametno poboljšava i svakodnevni život. Kamera telefona prevodi natpise i jelovnike na stranom jeziku i prikazuje prevod na samoj slici, a aplikacije za probu naočara ili šminke pokazuju kako će proizvod izgledati na licu korisnika. Posebno zanimljivu primenu, međutim, otvaraju pametna okruženja: savremene zgrade, kampusi i učionice opremljeni su senzorima koji neprekidno mere temperaturu, buku i kvalitet vazduha, ali ti podaci po pravilu ostaju skriveni u udaljenim kontrolnim tablama i izveštajima. Proširena stvarnost nudi daleko prirodniji pristup tim informacijama — dovoljno je uperiti telefon u vrata učionice da bi se iznad njih, u stvarnom prostoru, prikazali živi podaci o toj prostoriji: da li je zauzeta i do kada, koji se čas u njoj trenutno održava i kakvi su uslovi za rad. Upravo ta primena — prikaz kontekstualnih informacija iz pametnog okruženja u proširenoj stvarnosti — predmet je aplikacije opisane u ovom radu.

Razlog zbog kojeg je proširena stvarnost na pametnim telefonima doživela tako brzu ekspanziju jeste ukidanje ulaznih barijera. Korisniku nije potreban nijedan dodatni uređaj: dovoljan je telefon koji već poseduje, a programerima su na raspolaganju zrele softverske platforme koje složene probleme računarskog vida — praćenje položaja uređaja, prepoznavanje površina i procenu osvetljenja — rešavaju umesto njih. Na *Android* platformi centralno mesto među tim rešenjima zauzima *ARCore*, tehnologija kompanije *Google* na kojoj je zasnovana i aplikacija iz ovog rada, a koja je detaljno predstavljena u narednom poglavlju.

2 ARCore platforma

ARCore je platforma kompanije *Google* za razvoj aplikacija proširene stvarnosti na *Android* uređajima. Reč je o besplatnom kompletu za razvoj softvera (*SDK*) koji programerima stavlja na raspolaganje gotova rešenja za najteže probleme proširene stvarnosti: određivanje položaja uređaja u prostoru, prepoznavanje površina u okruženju i procenu osvetljenja scene. Platforma se na uređaje isporučuje kroz sistemsku komponentu *Google Play Services for AR*, koja se automatski ažurira putem prodavnice *Google Play*, pa aplikacije uvek koriste aktuelnu verziju bez potrebe da je same uključuju u instalacioni paket. *ARCore* je podržan na stotinama sertifikovanih modela telefona i tableta različitih proizvođača, pri čemu sertifikacija garantuje da su kamera, senzori pokreta i procesor uređaja dovoljno kvalitetni za stabilno iskustvo proširene stvarnosti. Zahvaljujući tome, jedna ista aplikacija radi na ogromnom broju uređaja koji se već nalaze u rukama korisnika, bez ikakvog dodatnog hardvera.

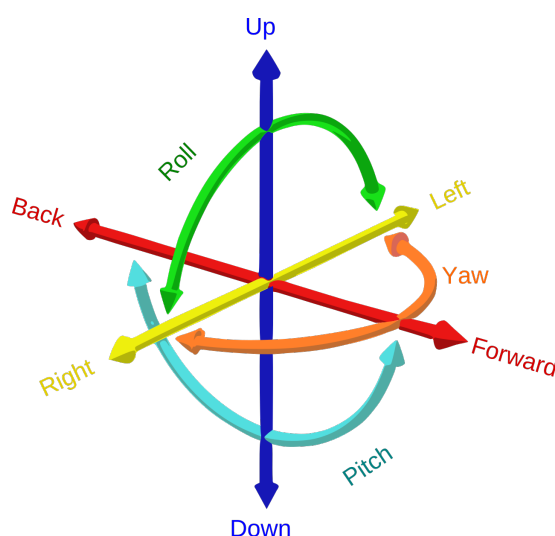
Sušтина platforme može se sažeti u tri temeljne sposobnosti: praćenje pokreta, razumevanje okruženja i procena osvetljenja. Svaka od njih rešava po jedan preduslov uverljive proširene stvarnosti, a zajedno omogućavaju da virtuelni sadržaj deluje kao da zaista postoji u fizičkom prostoru.



Slika 1: Logo platforme *ARCore*

2.1 Praćenje pokreta

Da bi virtuelni objekat ostao na svom mestu dok se korisnik kreće, telefon u svakom trenutku mora da zna gde se nalazi i kuda je usmeren. *ARCore* taj problem rešava postupkom koji se naziva vizuelno-inercijalna odometrija. Platforma na slici sa kamere pronalazi vizuelno upečatljive tačke — ivice, uglove i kontrastne detalje tekstura — i prati kako se njihov položaj menja iz kadra u kadar. Istovremeno očitava podatke sa akcelerometra i žiroskopa, senzora koji mere ubrzanje i rotaciju uređaja. Kombinovanjem ova dva izvora informacija, koji se međusobno dopunjuju i ispravljaju, *ARCore* izračunava položaj i orijentaciju telefona u prostoru sa šest stepeni slobode: tri translacije duž prostornih osa i tri rotacije oko njih. Zahvaljujući tome, virtuelni sadržaj se u svakom kadru iscrtava iz tačno odgovarajuće perspektive — kada korisnik priđe virtuelnom objektu, objekat se uvećava; kada ga zaobiđe, vidi ga sa druge strane, baš kao da je reč o stvarnom predmetu.

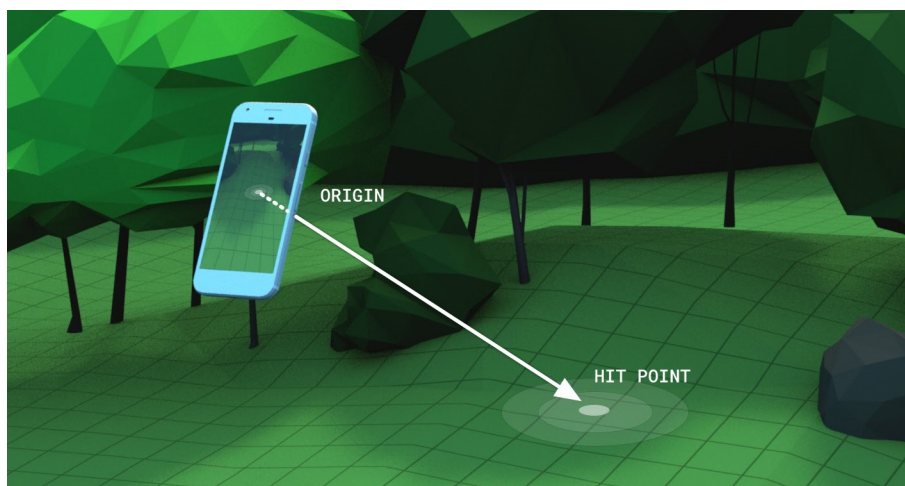


Slika 2: Šest stepeni slobode: tri translacije duž prostornih osa i tri rotacije oko njih

2.2 Razumevanje okruženja

Pored sopstvenog položaja, uređaj mora da razume i prostor oko sebe. *ARCore* neprekidno analizira raspored karakterističnih tačaka koje prati i, kada uoči grupe tačaka koje leže u istoj ravni, zaključuje da je pronašao površinu: pod, radni sto, sedište stolice ili zid. Platforma prepoznaje i horizontalne i vertikalne ravni, određuje njihove granice i proširuje ih kako kamera obuhvata nove delove prostora. Tako izgrađen model okruženja omogućava da se virtuelni objekti postavljaju na smisljena mesta — figura na pod, a slika na zid — umesto da lebde u vazduhu.

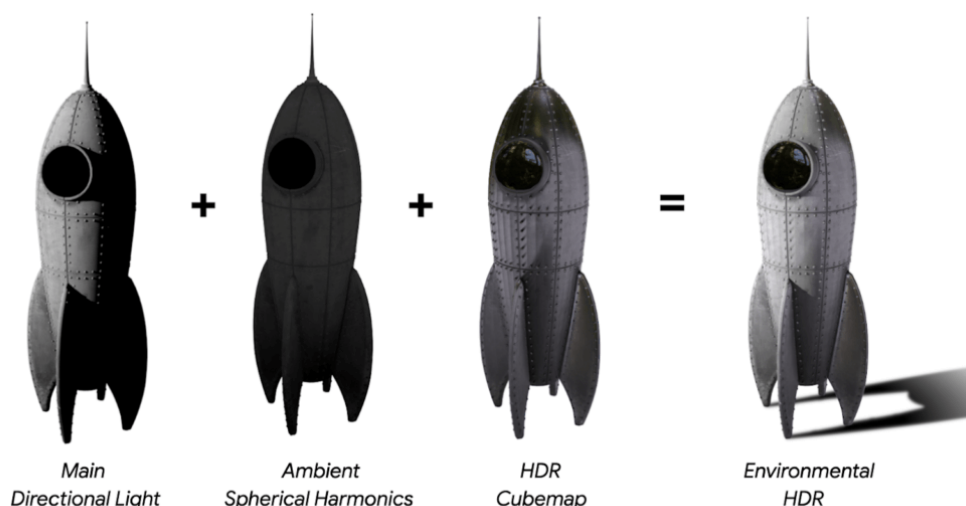
Važan mehanizam koji ovaj model čini upotrebljivim iz perspektive korisnika jeste testiranje pogodaka (engl. *hit-testing*). Kada korisnik dodirne ekran, *ARCore* iz položaja kamere kroz dodirnutu tačku na ekranu pušta zamišljeni zrak u trodimenzionalni prostor i izračunava gde taj zrak seče prepoznate ravni ili druge elemente okruženja. Rezultat je precizna tačka u prostoru, sa položajem i orijentacijom, na koju aplikacija može da postavi virtuelni sadržaj. Ovaj mehanizam je temelj velikog broja AR aplikacija, jer svaki dodir korisnika pretvara u tačku stvarnog prostora na koju se može vezati digitalni sadržaj.



Slika 3: Testiranje pogodaka: zrak iz kamere uređaja seče prepoznatu ravan u tački pogotka

2.3 Procena osvetljenja

Treća sposobnost platforme odnosi se na svetlost. Virtuelni objekat iscrtan bez obzira na uslove osvetljenja u prostoriji odmah odaje svoju veštačku prirodu: previše je svetao u mračnoj sobi ili bezličan na jakom suncu. *ARCore* zato iz slike sa kamere procenjuje karakteristike osvetljenja scene — prosečan intenzitet, boju svetlosti, a u režimu *Environmental HDR* i pravac glavnog izvora sveta te okolinsko osvetljenje pogodno za realistične odsjaje. Aplikacija te podatke koristi prilikom senčenja virtuelnih objekata, pa oni dobijaju senke i odsjaje usklađene sa stvarnim okruženjem, što značajno doprinosi utisku da objekat zaista pripada sceni.

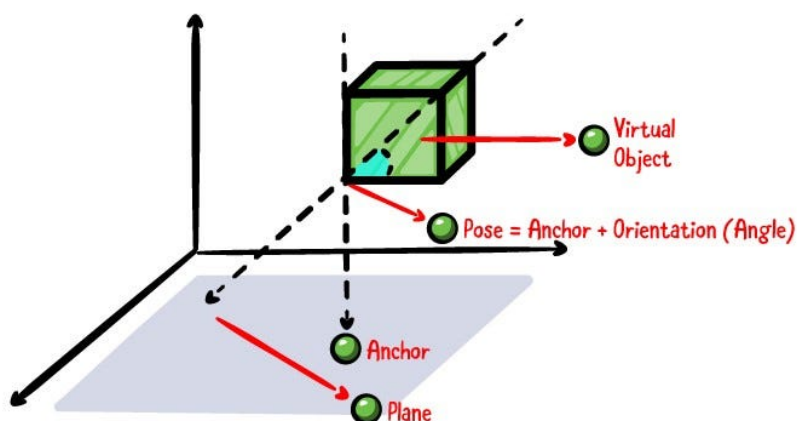


Slika 4: Komponente režima *Environmental HDR*: glavno direkciono svetlo, ambijentalno osvetljenje i *HDR* mapa okruženja zajedno daju realistično osenčen objekat

2.4 Dubina, sidra i objekti praćenja

Pored tri osnovne sposobnosti, za praktičan rad važno je još nekoliko pojmova. *Depth API* omogućava da *ARCore*, koristeći samo jednu običnu kameru, za svaki piksel slike proceni udaljenost od uređaja i tako izgradi dubinsku mapu scene. Zahvaljujući njoj, virtualni sadržaj se može postavljati na proizvoljne površine, a ne samo na prepoznate ravni, dok virtualni objekti mogu biti zaklonjeni stvarnim predmetima koji se nalaze ispred njih (okluzija), što dodatno pojačava uverljivost prikaza.

Virtualni sadržaj se za stvarni svet vezuje pomoću sidara (engl. *anchor*). Sidro predstavlja fiksnu pozu u prostoru koju platforma aktivno održava: kako *ARCore* vremenom prikuplja nove informacije i precizira svoje razumevanje scene, on koriguje i položaje sidara, pa sadržaj pričvršćen za njih ostaje „prikovan” za isto mesto u stvarnom svetu umesto da postepeno otpluta. Sidra se po pravilu vezuju za objekte praćenja (engl. *trackable*) — ravni, karakteristične tačke i druge elemente koje platforma prepoznaje i prati kroz vreme.



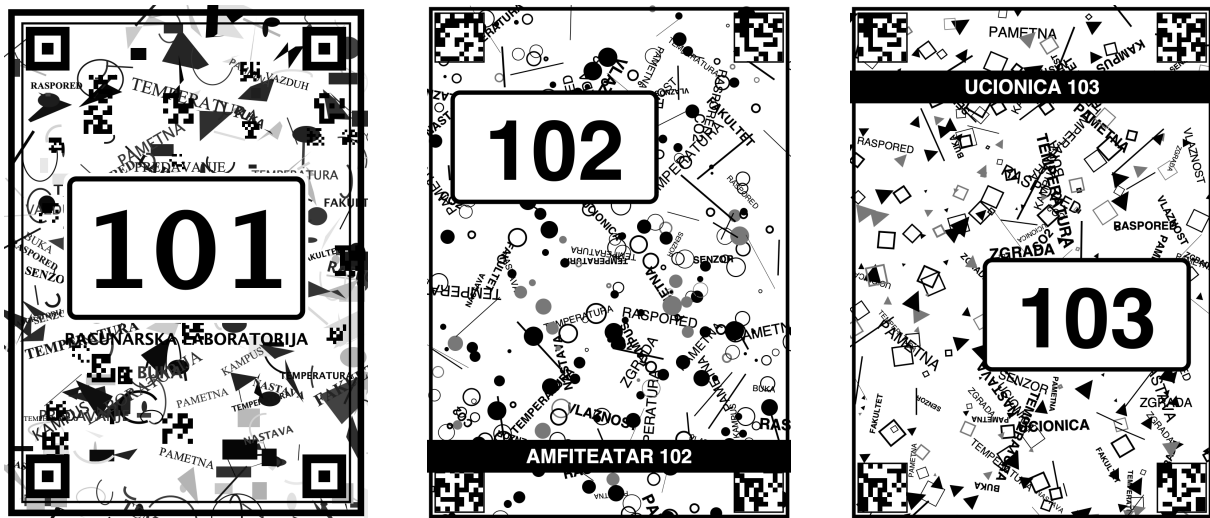
Slika 5: Sidro vezano za prepoznatu ravan: poza sidra (položaj i orijentacija) određuje gde se virtualni objekat iscrtava u prostoru

U praksi se *ARCore* koristi ili neposredno, kroz njegov izvorni programski interfejs, ili kroz biblioteke višeg nivoa koje pojednostavljaju iscrtavanje trodimenzionalnog sadržaja. Među njima se izdvaja *SceneView*, biblioteka koja objedinjuje *ARCore* i grafički pogon *Filament*, pa programer umesto niskog grafičkog koda radi sa scenom, čvorovima i modelima, što znatno ubrzava razvoj.

2.5 Praćenje slika (Augmented Images)

Pored ravni i karakterističnih tačaka, *ARCore* ume da prepoznaje i prati unapred zadate dvodimenzionalne slike u stvarnom svetu — sposobnost poznata pod nazivom *Augmented Images* (praćenje slika). Programer unapred sastavlja bazu slika, objekat tipa *AugmentedImageDatabase*, u koju svaku referentnu sliku dodaje pozivom *addImage* uz jedinstveno ime i, opciono, fizičku širinu odštampanog primerka izraženu u metrima. Kada je baza pridružena konfiguraciji sesije, *ARCore* u svakom kadru upoređuje karakteristične tačke slike sa kamerom sa tačkama slike iz baze. Čim prepozna neku od njih, platforma vraća objekat *AugmentedImage* koji nosi ime prepoznate slike, njenu pozu u prostoru (*centerPose*),

procenjene dimenzije ($extentX$ i $extentZ$) i stanje praćenja. Prepoznata slika time postaje punopravan objekat praćenja: za nju se može vezati sidro, a virtuelni sadržaj postavljen u njenom koordinatnom sistemu prati je dok se korisnik kreće.



Slika 6: Referentne slike markera učionica 101, 102 i 103 iz baze slika aplikacije

Kvalitet prepoznavanja presudno zavisi od same slike markera. Dobra referentna slika bogata je detaljima, ima visok kontrast i izbegava ponavljajuće šare i velike jednobojne površine, jer se prepoznavanje zasniva na rasporedu karakterističnih tačaka. *Google* uz platformu isporučuje alat *arccoreimg*, koji sliku ocenjuje na skali od 0 do 100; preporučuje se ocena od najmanje 75 da bi praćenje bilo pouzdano. Navođenje fizičke širine slike dodatno je važno: kada zna stvarne dimenzije markera, platforma odmah po prepoznavanju tačno procenjuje njegovu udaljenost i pozu, umesto da ih postepeno precizira posmatranjem iz više uglova. Jedna baza može da sadrži i do hiljadu slika, pri čemu se istovremeno prati do dvadeset prepoznatih markera.

Za aplikaciju iz ovog rada praćenje slika predstavlja prirodan izbor i središnji mehanizam čitavog rešenja. Odštampani marker na vratima učionice jednoznačno identifikuje prostoriju — ime prepoznate slike u sebi nosi oznaku učionice, pa aplikacija odmah zna od kog servera zatražiti podatke — dok sidro vezano za pozu slike obezbeđuje da informacioni panel ostane „prikovan” iznad vrata i kada se korisnik pomera, približava ili menja ugao gledanja. Povrh svega, platforma radi na širokom spektru postojećih uređaja bez dodatne opreme, čime aplikacija postaje dostupna najširem krugu korisnika. Način na koji se ove mogućnosti konkretno koriste u razvoju prikazan je u narednim poglavljima.

3 Arhitektura sistema

Rešenje opisano u ovom radu čine tri komponente koje zajedno simuliraju i prikazuju jedno pametno okruženje: serverska aplikacija koja igra ulogu pametne zgrade, veb kontrolna tabla kojom se tokom demonstracije menjaju vrednosti senzora i *Android* AR aplikacija kao glavni potrošač podataka. Tok podataka je jednosmeran i jednostavan: kontrolna tabla šalje nove vrednosti senzora serveru, server ih čuva i obogaćuje izvedenim informacijama, a AR aplikacija ih periodično preuzima i isertava u prostoru.

1. Prezenter na kontrolnoj tabli pomeri klizač ili promeni zauzetost učionice; promena se

posle kratkog zadržka šalje serveru.

2. Server proverava da li su vrednosti u dozvoljenim granicama, upisuje ih u memorijsko stanje i na svaki upit izračunava status i preporuku.
3. AR aplikacija, dok prati marker učionice, na svake tri sekunde preuzima stanje te učionice i osvežava panel usidren iznad markera.

3.1 Serverska aplikacija

Server je *Spring Boot* aplikacija (verzija 3.5, Java 21) koja simulira pametno okruženje. Stanje učionica drži u memoriji, u strukturi `ConcurrentHashMap`, bez baze podataka: pri pokretanju se učitavaju tri učionice (101, 102 i 103) sa početnim vrednostima temperature, buke i ugljen-dioksida, zauzetošću i dnevnim rasporedom časova. Celokupan izvedeni sadržaj računa se na serveru, pa su oba klijenta samo prikazi dobijenih podataka. REST interfejs čine tri krajnje tačke prikazane u tabeli 1.

Metoda	Putanja	Opis
GET	<code>/api/rooms</code>	lista svih učionica sa trenutnim stanjem
GET	<code>/api/rooms/{roomId}</code>	stanje jedne učionice; 404 za nepoznat identifikator
PUT	<code>/api/rooms/{roomId}/sensors</code>	upis novih vrednosti senzora i zauzetosti

Tabela 1: Krajnje tačke REST interfejsa servera

Za svaku izmerenu veličinu server izračunava status sa vrednostima OK, WARNING ili CRITICAL prema zadatim pragovima: temperatura je u redu između 20 i 26 °C, buka ispod 45 dB, a ugljen-dioksid ispod 800 ppm, dok vrednosti izvan širih granica postaju kritične. Na osnovu statusa formira se i preporuka na srpskom jeziku kao podrška odlučivanju — od „Uslovi su optimalni.” preko „Pratiti uslove u prostoriji.” do „Hitno provetriti prostoriju.” kada je koncentracija ugljen-dioksida kritična. Server iz rasporeda časova i sopstvenog sata izvodi i tekst zauzetosti (naziv tekućeg časa i vreme do kojeg je učionica zauzeta), a vraćenim vrednostima dodaje blagi sinusni šum kako bi paneli delovali živo i bez stalnih promena na kontrolnoj tabli.

Izgled kompletnog odgovora servera najbolje ilustruje stvaran primer. Na upit `GET /api/rooms/101` server vraća sledeći JSON dokument:

```

{
  "roomId": "101",
  "roomName": "Računarska laboratorija 101",
  "occupied": true,
  "currentClassName": "Veštačka inteligencija",
  "occupiedUntil": "14:00",
  "schedule": [
    {
      "startTime": "08:15",
      "endTime": "10:00",
      "className": "Pametna okruženja",
      "lecturerName": "prof. Maja Petrović"
    },
    {
      "startTime": "10:15",
      "endTime": "12:00",
      "className": "Mobilne aplikacije",
      "lecturerName": "prof. Nikola Janković"
    },
    {
      "startTime": "12:15",
      "endTime": "14:00",
      "className": "Veštačka inteligencija",
      "lecturerName": "prof. Ana Stojanović"
    }
  ],
  "temperatureCelsius": 23.3,
  "temperatureStatus": "OK",
  "noiseDecibels": 40.5,
  "noiseStatus": "OK",
  "carbonDioxidePpm": 735,
  "airQualityStatus": "OK",
  "recommendation": "Uslovi su optimalni."
}

```

Slika 7: Odgovor servera na upit GET /api/rooms/101

3.2 Kontrolna tabla

Kontrolna tabla je jednostrana *Angular* aplikacija (naziv kontrolna-tabla) sa po jednom karticom za svaku učionicu. Kartica sadrži prekidač zauzetosti i tri klizača — temperaturu, buku i ugljen-dioksid — čija se svaka promena, posle zadržka od 300 ms (engl. *debounce*), šalje serveru pozivom PUT /api/rooms/{roomId}/sensors. Tabla istovremeno na svakih pet sekundi preuzima sveže stanje svih učionica. Njena uloga u sistemu je demonstraciona: pošto u učionicama ne postoje stvarni senzori, prezenter preko klizača uživo simulira promene u pametnom okruženju i posmatra kako se one odražavaju na AR panelu.



Slika 8: Kontrolna tabla sa karticama učionica i klizačima

3.3 AR aplikacija

Treća i centralna komponenta je *Android* aplikacija zasnovana na *ARCore* platformi, glavni potrošač podataka iz pametnog okruženja. Ona prepoznaje marker na vratima učionice, iz njegovog imena izvodi identifikator prostorije, preuzima njeno stanje sa servera i iznad markera sidri panel sa živim podacima. Razvoju i implementaciji ove aplikacije posvećena su naredna dva poglavlja.



Slika 9: Logo AR aplikacije

4 Razvoj ARCore aplikacije

Pre implementacije samog AR pregleda učionica potrebno je pripremiti razvojno okruženje, obezbediti uređaj koji podržava ARCore i postaviti strukturu projekta sa neophodnim zavisnostima. U nastavku su opisani ti koraci, pri čemu navedene verzije alata i biblioteka odgovaraju stvarnom projektu iz ovog rada.

4.1 Potrebna oprema i softver na računaru

Razvoj se odvija u okruženju *Android Studio*, zvaničnom integrisanom razvojnom okruženju za Android platformu. Dovoljno je instalirati poslednju stabilnu verziju, jer ona već sadrži odgovarajući JDK (verzije 17 ili novije) i menadžer za *Android SDK*, kroz koji se preuzima platforma koja odgovara vrednosti `compileSdk = 36` iz ovog projekta. Programski jezik je *Kotlin* 2.1.20, u kombinaciji sa *Android Gradle Plugin*-om 8.13.0, dok se sam *Gradle* ne instalira posebno — projekat sadrži *Gradle wrapper*, skriptu koja pri prvom pokretanju preuzima tačno onu verziju alata sa kojom je projekat testiran.

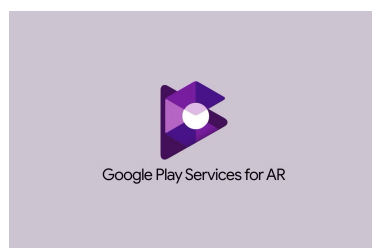


Slika 10: Alati korišćeni u razvoju: *Android Studio*, *Kotlin* i *Gradle*

Kada je reč o emulatoru, razvoj AR aplikacije realno zahteva fizički uređaj: emulator podržava ARCore samo u ograničenim scenarijima, sa virtuelnom scenom umesto stvarnog okruženja, pa se ponašanje aplikacije — posebno prepoznavanje odštampanog markera pravom kamerom i stabilnost sidrenja — ne može verodostojno proveriti bez pravog telefona.

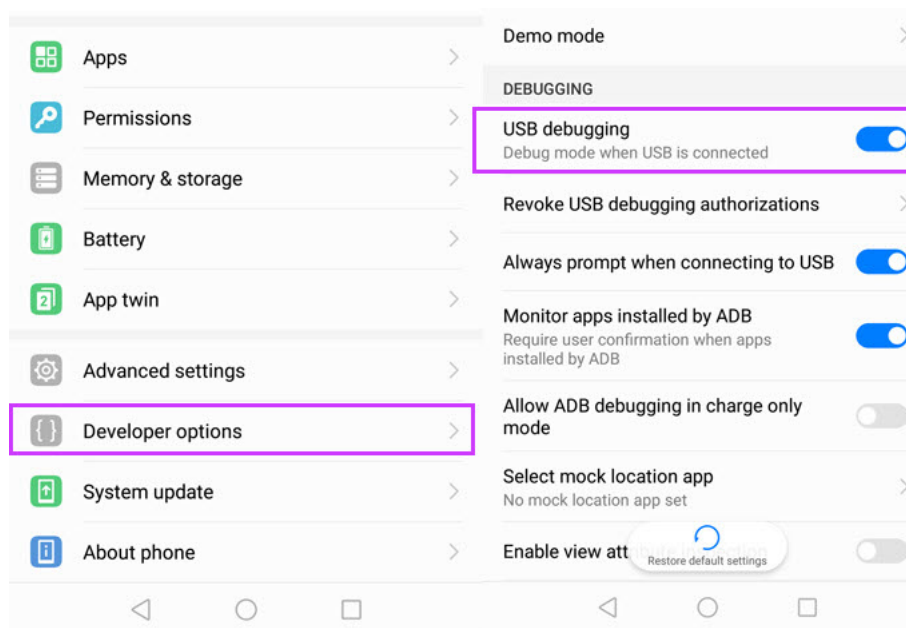
4.2 Potrebna oprema na telefonu

Za pokretanje aplikacije neophodan je Android uređaj sertifikovan za ARCore — Google održava javnu listu podržanih modela čije su kamere, senzori pokreta i procesori prošli proveru kvaliteta praćenja. Uređaj mora imati Android 7.0 (API nivo 24) ili noviji, što se poklapa sa vrednošću `minSdk = 24` u konfiguraciji projekta. Pored toga, na uređaju mora biti instalirana usluga *Google Play Services for AR* (ARCore), koja se distribuira i automatski ažurira preko prodavnice *Google Play*, pa aplikacija ne pakuje ARCore u sebe, već se oslanja na tu sistemsku uslugu.



Slika 11: Usluga *Google Play Services for AR*

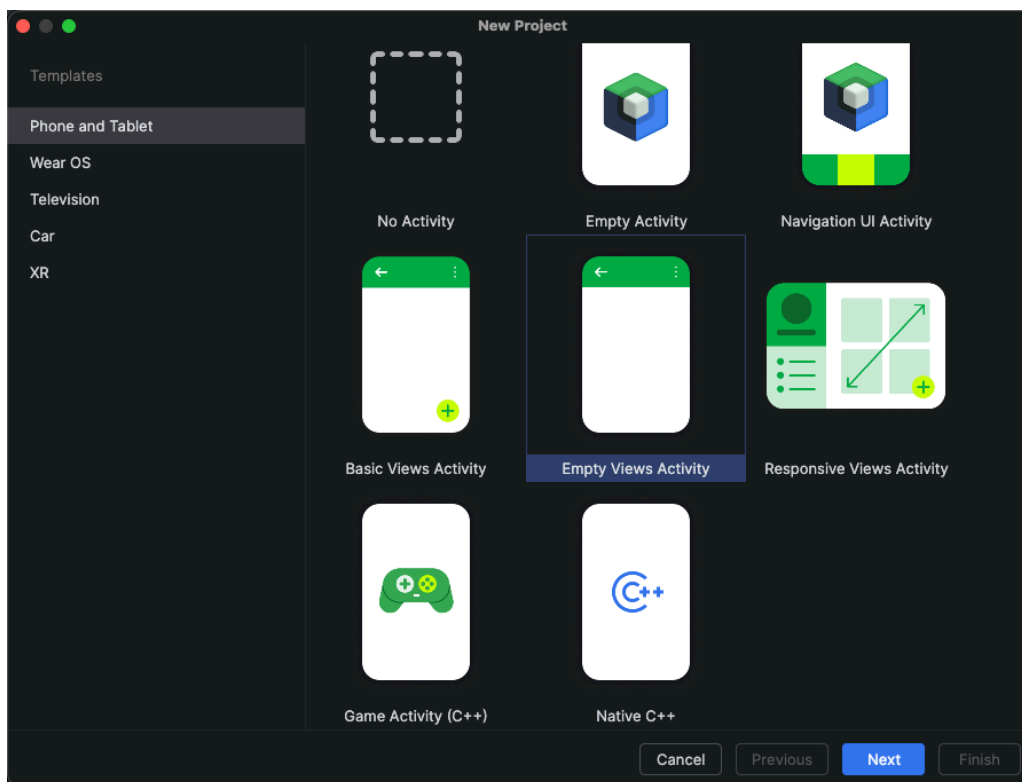
Da bi se aplikacija instalirala direktno iz okruženja *Android Studio*, na telefonu je potrebno uključiti opcije za programere i dozvoliti otklanjanje grešaka preko USB veze (*USB debugging*); alternativno, novije verzije okruženja podržavaju i bežično uparivanje preko Wi-Fi mreže. Pošto aplikacija podatke preuzima sa servera koji radi na računaru, veza se ostvaruje preko istog USB kabla: komanda `adb reverse tcp:8080 tcp:8080` preusmerava port 8080 telefona na računar, pa aplikacija server vidi na adresi `http://localhost:8080`.



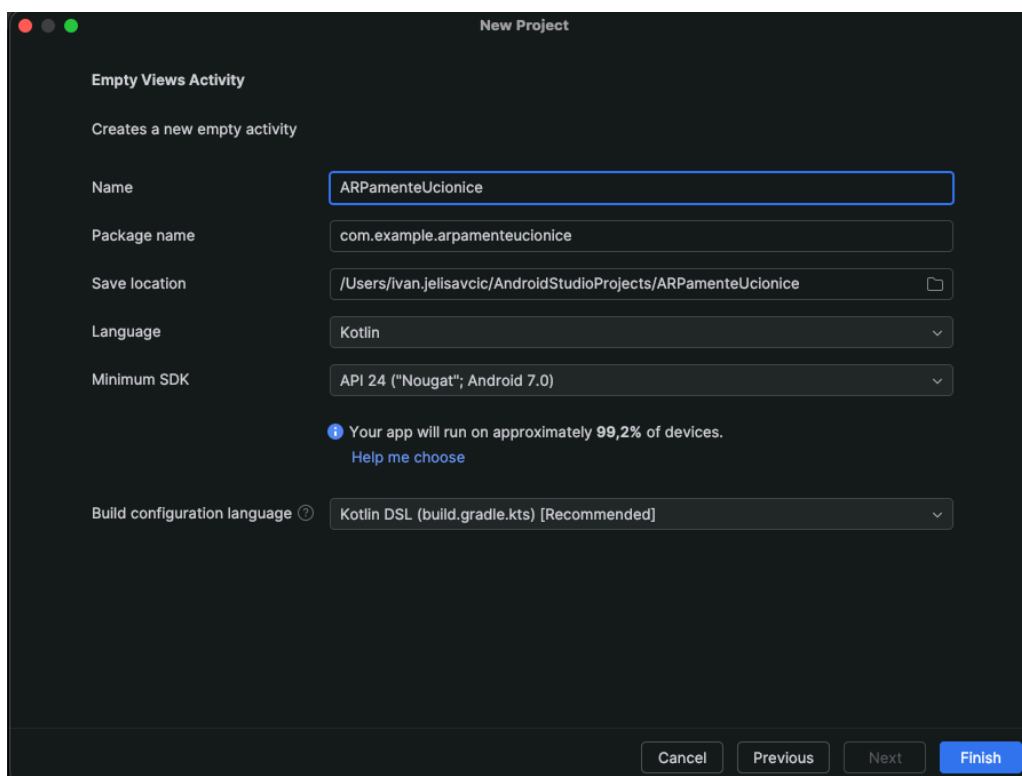
Slika 12: Uključivanje opcija za programere i otklanjanja grešaka preko USB veze (*USB debugging*) na telefonu

4.3 Kreiranje projekta i zavisnosti

Projekat se kreira kroz čarobnjak (engl. *wizard*) u okruženju *Android Studio*, izborom šablona *Empty Views Activity*, sa jezikom *Kotlin* i konfiguracijom izgradnje u obliku *Gradle Kotlin DSL* datoteka (`build.gradle.kts`). Nakon kreiranja, u datoteku `app/build.gradle.kts` dodaju se zavisnosti prikazane na slici 15, čije su verzije centralizovane u katalogu `gradle/libs.versions.toml`.



Slika 13: Čarobnjak za kreiranje projekta sa izabranim šablonom *Empty Views Activity*



Slika 14: Podešavanja novog projekta: jezik *Kotlin*, minimalni SDK API 24 i konfiguracija izgradnje *Kotlin DSL* (build.gradle.kts)

```

dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.appcompat)
    implementation(libs.material)
    implementation(libs.androidx.constraintlayout)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.arsceneview)
    implementation(libs.retrofit)
    implementation(libs.retrofit.converter.kotlinx.serialization)
    implementation(libs.kotlinx.serialization.json)
    implementation(libs.okhttp.logging.interceptor)
}

```

Slika 15: Zavisnosti korišćene u projektu, blok dependencies u datoteci app/build.gradle.kts

Ključna zavisnost je biblioteka *SceneView* (arsceneview), koja objedinjuje ARCore i *Filament* mehanizam za renderovanje: umesto direktnog korišćenja ARCore-a, što bi zahtevalo ručno pisanje OpenGL koda za prikaz kamere i 3D objekata, SceneView nudi gotovu komponentu ARSceneView, graf scene sa čvorovima (Node) i renderovanje „iz kutije”. Za mrežnu komunikaciju sa serverom koristi se *Retrofit* u kombinaciji sa *kotlinx-serialization* konvertorom, čime se JSON odgovori servera automatski pretvaraju u tipizirane Kotlin objekte, dok se *OkHttp logging-interceptor* uključuje samo u razvojnoj varijanti radi uvida u mrežni saobraćaj. Korutine i *lifecycle-runtime-ktx* obezbeđuju asinhrono periodično preuzimanje podataka usklađeno sa životnim ciklusom aktivnosti. U bloku android uključena je i opcija `buildFeatures { viewBinding = true }`, kojom se za svaki XML raspored generiše tipizovana klasa za pristup elementima interfejsa, čime se izbegavaju pozivi `findViewById`.

4.4 Podešavanje manifesta

Datoteka `AndroidManifest.xml` mora precizno opisati zahteve aplikacije prema sistemu. Projekat deklariše sledeće stavke:

- Dozvola za pristup kameri, koja je obavezna, jer bez kamere proširena stvarnost nije moguća, i dozvola za pristup internetu, neophodna za preuzimanje podataka sa servera:

```

<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.INTERNET" />

```

Slika 16: Dozvole za pristup kameri i internetu u datoteci `AndroidManifest.xml`

- Deklaracija da je AR hardver obavezan, u kombinaciji sa oznakom da je ARCore neizostavna komponenta aplikacije:

```
<uses-feature
    android:name="android.hardware.camera.ar"
    android:required="true" />
```

```
<meta-data
    android:name="com.google.ar.core"
    android:value="required"
    tools:replace="android:value" />
```

Slika 17: Deklaracija obaveznog AR hardvera i *ARCore* meta-oznaka u manifestu

Ovim parom aplikacija postaje „AR Required”, što znači da se u prodavnici nudi samo uređajima koji podržavaju ARCore. Oznaka `tools:replace` nad atributom `android:value` ovde je neophodna: biblioteka `SceneView` u svom manifestu istu meta-oznaku deklarise sa vrednošću `optional`, pa bi spajanje manifesta (*manifest merge*) bez ove oznake završilo greškom zbog sukoba vrednosti; ovako vrednost iz aplikacije nadjačava vrednost iz biblioteke.

- Atribut `android:usesCleartextTraffic="true"` na elementu `<application>`, kojim se dozvoljava nešifrovan HTTP saobraćaj. Od Android-a 9 sistem podrazumevano blokira saobraćaj bez TLS-a, a server iz ovog rada dostupan je telefonu na adresi `http://localhost:8080`, bez sertifikata, pa je izuzetak neophodan za demonstraciju.

4.5 Koraci do prve AR scene

Kada su okruženje, zavisnosti i manifest spremni, do prve funkcionalne AR scene dolazi se kroz nekoliko koraka:

1. U XML raspored aktivnosti dodaje se komponenta `ARSceneView`, koja istovremeno prikazuje sliku sa kamere i renderovane 3D objekte.

```
<io.github.sceneview.ar.ARSceneView
    android:id="@+id/sceneView"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Slika 18: Komponenta `ARSceneView` u XML rasporedu aktivnosti

2. Pogled na scenu povezuje se sa životnim ciklusom aktivnosti dodelom `sceneView.lifecycle = lifecycle`, čime se AR sesija automatski pauzira i nastavlja zajedno sa aktivnošću.


```
binding.sceneView.apply {
    lifecycle = this@AugmentedRealityActivity.lifecycle
}
```

Slika 19: Vezivanje pogleda na scenu za životni ciklus aktivnosti

3. U pozivu `configureSession` konfiguriše se sesija: gradi se baza slika `AugmentedImageDatabase` sa markerima učionica i dodeljuje polju `config.augmentedImageDatabase`, čime se uključuje praćenje slika opisano u odeljku 2.5.

```
configureSession { session, config ->
    val augmentedImageDatabase = AugmentedImageDatabase(session)
    val markerFileNames = assets.list( path = "markers").orEmpty()
        .filter { it.endsWith( suffix = ".png") }
    markerFileNames.forEach { markerFileName ->
        val imageName = markerFileName.substringBeforeLast( delimiter = '.')
        try {
            assets.open( fileName = "markers/$markerFileName").use { markerStream ->
                val markerBitmap = BitmapFactory.decodeStream( is = markerStream)
                if (markerBitmap != null) {
                    augmentedImageDatabase.addImage(
                        imageName,
                        markerBitmap,
                        widthInMeters = MARKER_PHYSICAL_WIDTH_METERS
                    )
                }
            }
        } catch (exception: Exception) {
            Log.w(
                tag = "AugmentedRealityActivity",
                msg = "Preskočen marker $markerFileName",
                tr = exception
            )
        }
    }
    config.augmentedImageDatabase = augmentedImageDatabase
}
```

Slika 20: Konfiguracija sesije i izgradnja baze slika markera

4. Podrazumevani prikaz detektovanih ravni isključuje se postavkom svojstva `isEnabled` komponente `planeRenderer` na vrednost netačno, jer aplikaciji ravni nisu potrebne, a istaknute površine bi vizuelno opterećivale ekran.

```
planeRenderer.isEnabled = false
```

Slika 21: Isključivanje prikaza detektovanih ravni

5. Preko povratnog poziva `onSessionUpdated` aplikacija se uključuje u obradu svakog frejma i iz njega čita novoprepoznate i ažurirane slike markera.

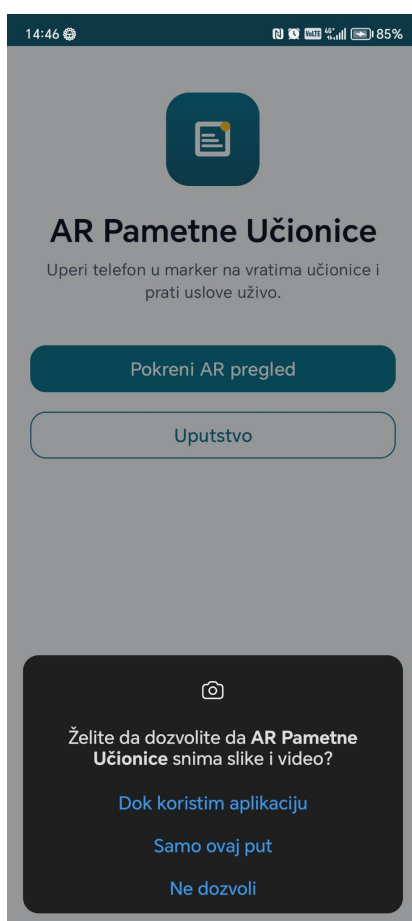
```

onSessionUpdated = { _, frame ->
  frame.getUpdatedAugmentedImages().forEach { augmentedImage ->
    when {
      augmentedImage.isTracking -> onAugmentedImageTracking(augmentedImage)
      augmentedImage.trackingState == TrackingState.STOPPED ->
        removePanel( imageName = augmentedImage.name)
    }
  }
}
}

```

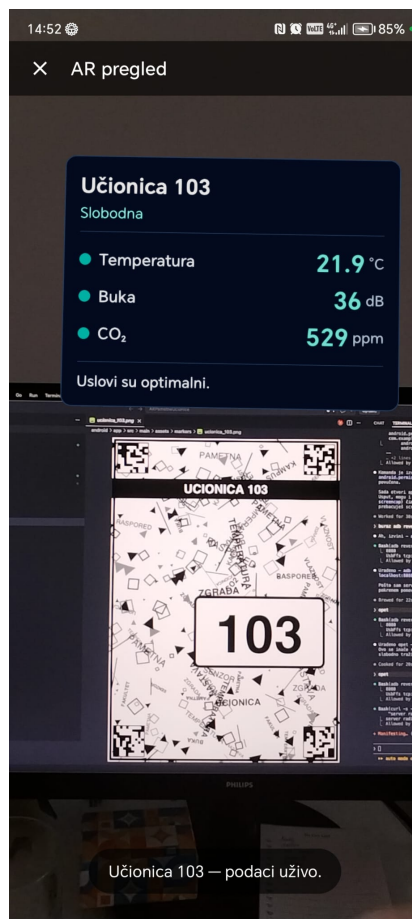
Slika 22: Obrada prepoznatih slika markera u povratnom pozivu onSessionUpdated

6. U vreme izvršavanja traži se dozvola za kameru, budući da se od API nivoa 23 opasne dozvole moraju eksplicitno odobriti od strane korisnika.



Slika 23: Sistemski dijalog za dozvolu pristupa kameri

7. Aplikacija se pokreće na fizičkom uređaju, sa odštampanim markerom, gde se proverava da li se marker prepoznaje i da li panel stabilno stoji iznad njega.



Slika 24: Prva AR scena: panel usidren iznad prepoznatog markera

U narednom poglavlju prikazano je kako se od ovih osnovnih gradivnih elemenata — sesije, baze slika, sidara i čvorova — gradi kompletna funkcionalnost AR pregleda učionica.

5 Implementacija AR pregleda učionica

Centralna komponenta aplikacije je aktivnost `AugmentedRealityActivity`, u kojoj je smeštena celokupna AR logika. Aktivnost koristi komponentu `ARSceneView` iz biblioteke `SceneView`, koja objedinjuje AR sesiju biblioteke `ARCore` i grafički prikaz virtuelnih objekata preko slike sa kamere. Pošto u jednoj sesiji može istovremeno da bude praćeno više učionica, aktivnost sve resurse vodi u mapama ključanim imenom prepoznate slike: mapu sidrišnih čvorova, mapu panela i mapu poslova za periodično preuzimanje podataka. U nastavku je opisano kako aplikacija koristi ključne funkcije `ARCore` i `SceneView` interfejsa: od konfiguracije sesije i baze markera, preko prepoznavanja učionice i sidrenja panela, do iscrtavanja podataka i oslobađanja resursa.

5.1 Konfiguracija sesije i baze slika

Baza referentnih slika gradi se u pozivu `configureSession`, pri svakom pokretanju AR sesije. Aplikacija iz fascikle `assets/markers` čita sve PNG datoteke i svaku dodaje u bazu pod imenom koje odgovara nazivu datoteke bez ekstenzije:

```

configureSession { session, config ->
    val augmentedImageDatabase = AugmentedImageDatabase(session)
    val markerFileNames = assets.list( path = "markers").orEmpty()
        .filter { it.endsWith( suffix = ".png") }
    markerFileNames.forEach { markerFileName ->
        val imageName = markerFileName.substringBeforeLast( delimiter = '.')
        try {
            assets.open( fileName = "markers/$markerFileName").use { markerStream ->
                val markerBitmap = BitmapFactory.decodeStream( is = markerStream)
                if (markerBitmap != null) {
                    augmentedImageDatabase.addImage(
                        imageName,
                        markerBitmap,
                        widthInMeters = MARKER_PHYSICAL_WIDTH_METERS
                    )
                }
            }
        } catch (exception: Exception) {
            Log.w(
                tag = "AugmentedRealityActivity",
                msg = "Preskočen marker $markerFileName",
                tr = exception
            )
        }
    }
    config.augmentedImageDatabase = augmentedImageDatabase
}

```

Slika 25: Izgradnja baze slika markera u pozivu `configureSession`

Treći parametar funkcije `addImage` je fizička širina odštampanog markera, konstanta `MARKER_PHYSICAL_WIDTH_METERS` vrednosti `0.21f` — marker se štampa na papiru formata A4, čija je širina 21 centimetar. Zahvaljujući tome ARCore odmah po prepoznavanju tačno procenjuje pozu i udaljenost markera. Svaki poziv `addImage` obavljen je u `try/catch` blok: ako neka slika nema dovoljno karakterističnih tačaka, ARCore odbija da je upiše u bazu izuzetkom, pa se takva slika preskače uz upis u dnevnik umesto da obori čitavu sesiju. Ako nijedna slika ne uđe u bazu, korisniku se ispisuje poruka „Nema markera u bazi aplikacije.”. Pored konfiguracije sesije, isključen je i podrazumevani prikaz detektovanih ravni postavkom `planeRenderer.isEnabled = false`, jer aplikacija ravni ne koristi.

5.2 Prepoznavanje učionice

Preko povratnog poziva `onSessionUpdated` aplikacija se uključuje u obradu svakog frejma AR sesije. Iz frejma se funkcijom `getUpdatedAugmentedImages` čitaju slike čije se stanje u tom frejmu promenilo:

```

onSessionUpdated = { _, frame ->
    frame.getUpdatedAugmentedImages().forEach { augmentedImage ->
        when {
            augmentedImage.isTracking -> onAugmentedImageTracking(augmentedImage)
            augmentedImage.trackingState == TrackingState.STOPPED ->
                removePanel( imageName = augmentedImage.name)
        }
    }
}

```

Slika 26: Obrada ažuriranih slika markera u povratnom pozivu onSessionUpdated

Slika u stanju TRACKING aktivno se prati i njena poza je pouzdana, pa se za nju postavlja panel; stanje STOPPED znači da je praćenje trajno prekinuto, pa se svi resursi vezani za tu sliku oslobađaju. Identifikator učionice izvodi se iz imena prepoznate slike: marker ucionica_101 daje identifikator 101 izrazom `imageName.substringAfterLast('_')`. Taj identifikator se zatim koristi u putanji serverskog poziva, a korisniku se u statusnoj traci ispisuje poruka „Učionica 101 — podaci uživo.”.

5.3 Sidrenje panela

Kada slika prvi put uđe u stanje praćenja, aplikacija za nju kreira sidro u centru markera i obavlja ga u čvor scene:

```

val anchor = augmentedImage.createAnchor(augmentedImage.centerPose)
val anchorNode = AnchorNode(binding.sceneView.engine, anchor)
anchorNode.isPositionEditable = false
binding.sceneView.addChildNode(anchorNode)
trackedAnchors[imageName] = anchorNode
panelOffsets[imageName] =
    -(augmentedImage.extentZ / 2f + PANEL_BOTTOM_GAP_METERS + PANEL_HEIGHT_METERS / 2f)

```

Slika 27: Kreiranje sidra i sidrišnog čvora za prepoznati marker

Sidro vezano za pozu slike obezbeđuje da sadržaj ostane pričvršćen za marker dok ARCore precizira svoje razumevanje scene. Sam panel je čvor tipa `ImageNode` — ravan pravougaonik na koji se kao tekstura preslikava bitmapa — veličine 32 puta 24 centimetra u stvarnom prostoru:

```

val panelNode = ImageNode(
    materialLoader = binding.sceneView.materialLoader,
    bitmap = panelBitmap,
    size = Size(x = PANEL_WIDTH_METERS, y = PANEL_HEIGHT_METERS, z = 0f)
)
panelNode.rotation = Rotation(x = -90f, y = 0f, z = 0f)
panelNode.position = Position(x = 0f, y = 0f, z = panelOffsets[imageName] ?: DEFAULT_PANEL_OFFSET_METERS)
anchorNode.addChildNode(panelNode)
panelNodes[imageName] = panelNode

```

Slika 28: Kreiranje i pozicioniranje čvora panela `ImageNode`

Koordinatni sistem prepoznate slike postavlja X osu duž njene širine, Z osu duž visine, a Y osu upravno na nju, pa se rotacijom od -90 stepeni oko X ose pravougaonik panela poravnava sa ravni markera. Pomeraj duž negativne Z ose, izračunat prilikom prepoznavanja, iznosi polovinu visine markera ($\text{extentZ} / 2$) uvećanu za razmak od tri centimetra i polovinu visine samog panela — time panel lebdi tačno iznad gornje ivice markera, odnosno iznad vrata učionice.

5.4 Iscrtavanje panela

Sadržaj panela ne modeluje se kao 3D geometrija, već se koristi postojeći sistem Android rasporeda: klasa `PanelBitmapRenderer` naduvava XML raspored `view_room_panel.xml`, popunjava ga podacima i iscrtava u bitmapu. Panel prikazuje naziv učionice, red zauzetosti („Zauzeta do 10:00 — Pametna okruženja” ili „Slobodna”), tri reda sa vrednostima temperature, buke i ugljen-dioksida uz kružne indikatore obojene prema statusu (zeleno za OK, žuto za WARNING, crveno za CRITICAL) i red sa preporukom servera, koji se pri kritičnom statusu takođe boji crveno. Popunjeni raspored se zatim meri i iscrtava u fiksnoj rezoluciji:

```
val panelView = binding.root
panelView.measure(
    widthMeasureSpec = View.MeasureSpec.makeMeasureSpec( size = BITMAP_WIDTH, mode = View.MeasureSpec.EXACTLY),
    heightMeasureSpec = View.MeasureSpec.makeMeasureSpec( size = BITMAP_HEIGHT, mode = View.MeasureSpec.EXACTLY)
)
panelView.layout( l = 0, t = 0, r = BITMAP_WIDTH, b = BITMAP_HEIGHT)

val bitmap = Bitmap.createBitmap(BITMAP_WIDTH, BITMAP_HEIGHT, config = Bitmap.Config.ARGB_8888)
panelView.draw(Canvas(bitmap))
```

Slika 29: Iscrtavanje rasporeda panela u bitmapu fiksne rezolucije

Konstante `BITMAP_WIDTH` i `BITMAP_HEIGHT` iznose 1024 i 768 piksela, što pri odnosu stranica 4:3 odgovara fizičkim dimenzijama panela od 32 puta 24 centimetra. Kada za već prikazanu učionicu stignu novi podaci, čvor se ne stvara iznova: dovoljno je postojećem `ImageNode` objektu dodeliti novu vrednost svojstva `bitmap`, čime `SceneView` zamenjuje teksturu na mestu, bez trzaja u sceni.

5.5 Preuzimanje podataka u realnom vremenu

Komunikacija sa serverom izvedena je bibliotekom *Retrofit* uz *kotlinx-serialization* konvertor. Mrežni interfejs sadrži dve suspendovane funkcije:

```

4 Usages
interface RoomApiService {

    @GET( value = "api/rooms")
    suspend fun getRooms(): List<RoomResponse>

    1 Usage
    @GET( value = "api/rooms/{roomId}")
    suspend fun getRoom(@Path( value = "roomId") roomId: String): RoomResponse
}

```

Slika 30: Mrežni interfejs RoomApiService sa suspendovanim funkcijama

Odgovori servera preslikavaju se u tipizirane objekte: klasa RoomResponse označena anotacijom @Serializable sadrži tačno ona polja koja server vraća (naziv, zauzetost, raspored, vrednosti senzora, statuse i preporuku), pri čemu su statusi modelovani nabranjanjem SensorStatus sa vrednostima OK, WARNING i CRITICAL. Adresa servera zadata je u kodu vrednošću `http://localhost:8080`: telefon je tokom razvoja i demonstracije povezan sa računarom USB kablom, a komanda `adb reverse tcp:8080 tcp:8080` preusmerava taj port telefona na server koji radi na računaru.

Za svaku praćenu učionicu pokreće se po jedan posao (Job) u opsegu lifecycleScope, koji u petlji preuzima stanje i osvežava panel na svake tri sekunde:

```

pollingJobs.remove( key = imageName)?.cancel()
pollingJobs[imageName] = lifecycleScope.launch {
    while (isActive) {
        try {
            val room = roomApiService.getRoom(roomId)
            val panelBitmap =
                panelBitmapRenderer.render( context = this@AugmentedRealityActivity, room)
            updatePanel(imageName, panelBitmap)
            setStatusText("Učionica $roomId - podaci uživo.")
        } catch (exception: IOException) {
            setStatusText("Server nije dostupan. Proveri adresu servera.")
        } catch (exception: HttpException) {
            setStatusText("Server nije dostupan. Proveri adresu servera.")
        }
        delay( timeMillis = POLLING_INTERVAL_MILLISECONDS)
    }
}
}

```

Slika 31: Petlja periodičnog preuzimanja podataka i osvežavanja panela

Mrežne greške ne prekidaju petlju: korisniku se ispisuje poruka o nedostupnom serveru, a sledeća iteracija ponovo pokušava preuzimanje, pa se panel sam oporavlja čim server postane dostupan. Posao se otkazuje čim praćenje slike prestane, kao i u metodi `onPause`, čime se sprečava mrežni saobraćaj dok je aplikacija u pozadini; u metodi `onResume` poslovi se ponovo pokreću za sve učionice koje su i dalje usidrene.

5.6 Životni ciklus i oslobađanje resursa

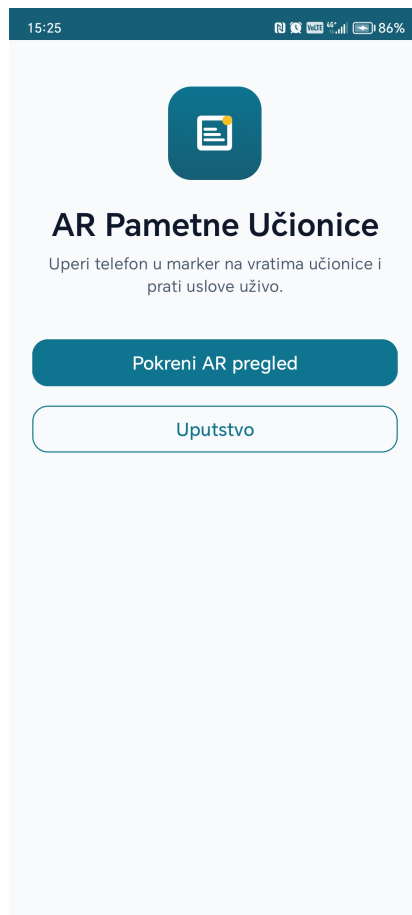
Kada ARCore praćenje neke slike pređe u stanje `STOPPED`, nije dovoljno samo sakriti panel: čvor panela se uklanja iz grafa scene i uništava pozivom `destroy()`, a zatim se isto čini i sa sidrišnim čvorom, čime se oslobađa i samo ARCore sidro. Ovaj korak je važan, jer svako aktivno sidro troši resurse praćenja u svakom frejmu — ARCore njegovu pozu neprestano ažurira — pa bi nagomilavanje neoslobođenih sidara vremenom degradiralo performanse sesije. Istim redosledom oslobađaju se i pripadajuće stavke u mapama i otkazuje posao za preuzimanje podataka, čime se garantuje da po prestanku praćenja ne ostaje nijedan aktivan resurs vezan za tu učionicu.

6 Prikaz aplikacije

U ovom poglavlju aplikacija je predstavljena kroz prikaz njenih ekrana, redosledom kojim se korisnik sa njima susreće pri tipičnoj upotrebi. Za svaki ekran dat je kratak opis sadržaja i mogućih interakcija, praćen snimkom ekrana. Korisnički interfejs izveden je u skladu sa Material dizajn (engl. *Material Design*) smernicama, uz upotrebu komponenti biblioteke `Material Components` za Android.

6.1 Početni ekran

Početni ekran predstavlja ulaznu tačku aplikacije. U gornjem delu prikazani su logotip u zaobljenom bloku, naziv AR Pametne Učionice i kratak opis koji korisniku objašnjava da se usmeravanjem kamere na marker učionice iznad njega prikazuju živi podaci o prostoriji. Ispod opisa nalaze se dva dugmeta: glavno dugme `Pokreni AR pregled` otvara AR ekran, dok dugme `Uputstvo` vodi na ekran sa uputstvom.



Slika 32: Početni ekran aplikacije

6.2 Ekran sa uputstvom

Ekran sa uputstvom sadrži četiri numerisana koraka prikazana u vidu zasebnih kartica. Prvi korak upućuje korisnika da pronađe marker na vratima učionice, drugi da kameru telefona usmeri ka markeru, treći objašnjava da se iznad markera prikazuje panel sa živim podacima o prostoriji, a četvrti da boje indikatora pokazuju status izmerenih vrednosti — zelena da je sve u redu, žuta upozorenje, a crvena kritično stanje.



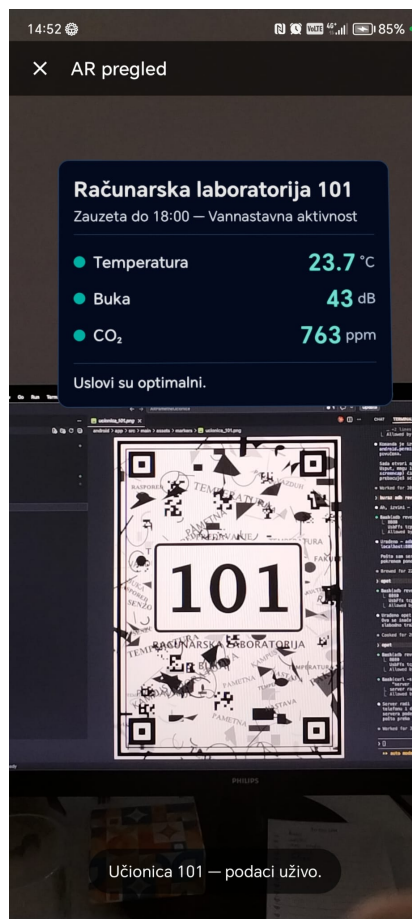
Slika 33: Ekran sa uputstvom za korišćenje

6.3 AR ekran — prepoznavanje markera

Nakon pokretanja AR pregleda otvara se ekran na kome slika sa kamere ispunjava celu površinu prikaza. Na vrhu se nalazi tamna traka sa dugmetom za zatvaranje i naslovom aplikacije, a ispod nje statusna poruka u obliku pilule koja u početnom stanju glasi `Pronađi marker učionice.` i koja korisnika tokom rada vodi kroz proces. U ovoj fazi korisnik polako pomera telefon dok kamera ne obuhvati odštampani marker na vratima učionice.

6.4 AR ekran — usidreni panel

Čim ARCore prepozna marker, iznad njegove gornje ivice pojavljuje se usidreni panel sa podacima o učionici, a statusna poruka se menja u `Učionica 101 — podaci uživo..` Panel prikazuje naziv prostorije, red zauzetosti sa nazivom tekućeg časa i vremenom do kojeg je učionica zauzeta, tri reda sa temperaturom, bukom i ugljen-dioksidom uz indikatore obojene prema statusu i preporuku servera. Panel ostaje pričvršćen za marker dok se korisnik kreće: može mu se prići, odmaći se ili ga posmatrati sa strane, a sadržaj se osvežava na svake tri sekunde svežim podacima sa servera.



Slika 34: Usidreni panel iznad markera sa živim podacima

6.5 Demonstracija sa kontrolnom tablom

Završni deo demonstracije povezuje sve tri komponente sistema. Prezenter na kontrolnoj tabli pomeri klizač — na primer, podigne vrednost ugljen-dioksida iznad kritičnog praga — i promena se posle zadržka od 300 milisekundi šalje serveru. Već pri sledećem preuzimanju, najkasnije tri sekunde kasnije, AR panel na telefonu prikazuje novu vrednost, indikator kvaliteta vazduha prelazi u crveno i ispisuje se preporuka *Hitno provetriti prostoriju.*, bez ijedne akcije na samom telefonu.

Prikazani ekrani zajedno čine zaokruženu celinu: od početnog ekrana i uputstva, preko prepoznavanja markera, do živog panela koji u realnom vremenu odražava stanje pametnog okruženja. Demonstracioni tok — klizač na kontrolnoj tabli, upis na serveru, osvežen panel u proširenoj stvarnosti u roku od tri sekunde — pokazuje da je arhitektura spremna i za stvarnu primenu: dovoljno je da umesto kontrolne table iste krajnje tačke pozivaju pravi senzori ugrađeni u učionice, bez ijedne izmene u AR aplikaciji. Kao moguća proširenja u budućem radu izdvajaju se povezivanje stvarnih senzora temperature, buke i ugljen-dioksida preko postojećeg interfejsa, dodavanje markera za veći broj učionica i zgrada, kao i interakcija na samom panelu, poput rezervacije slobodne učionice dodirom.